

Sigantry

Tutorials — Learn by Doing

12 worked, end-to-end examples you can follow verbatim — from first install to drift detection, rollback, scheduled alerts, and governance sweeps. Every command was executed against a live Fabric tenant; expected-output blocks show what you should actually see.

01	02	03	04	05	06	07	08	09	10
----	----	----	----	----	----	----	----	----	----

Release	3.2.1
Document date	14 June 2026
Audience	Operators & consumers, hands-on
Companion	Sigantry User Guide (concepts & reference)
License	Apache-2.0

Table of Contents

Tutorials — Learn Sigantry by Doing	6
Learning path	6
The tutorials	8
Conventions used in every tutorial	8
Prerequisites common to all tutorials	9
Related reading	9
Tutorial 01 — Setup and First Contact	10
What you need before starting	10
How the pieces fit	10
Step 1 — Install	12
Step 2 — Authenticate	12
Aside — running headless or as a service principal (optional)	12
Step 3 — Health check the install	13
Step 4 — First contact: list your workspaces	13
Step 5 — Snapshot it	14
What just happened	14
Troubleshooting	14
Success checklist	14
Tutorial 02 — Sync Notebooks into a Workspace	16
The mental model	16
Step 1 — Make a working folder with two notebooks	18
Step 2 — Author the manifest	18
Step 3 — Author a minimal parameters.yml	18
Step 4 — Dry-run	19
Step 5 — Apply with first-time publish	19
Step 6 — Prove idempotency	19
What just happened — and what deliberately did NOT	19
Troubleshooting	20
Success checklist	20
Tutorial 03 — Detect Drift	21
What drift means here	21
Step 1 — Baseline: prove you currently have no drift	22
Step 2 — Inject drift from the portal	22
Step 3 — Catch it	22
Step 4 — Reconcile	22
Step 5 — Confirm the clean state	23
Caveat that will save you a bad on-call page	23
Success checklist	23
Tutorial 04 — Read the Audit Trail	24
Where the evidence lives	24
Step 1 — List your releases	25

Step 2 — Inspect one record	25
Step 3 — Diff two releases	25
Step 4 — Verify integrity, then break it on purpose	25
Step 5 — (Optional) link a release to a work item	26
A note on ledger hygiene	26
Success checklist	26
Tutorial 05 — Adopt an Existing Workspace (Brownfield)	28
Why this matters	28
Step 1 — Pull	29
Step 2 — Read what you got	29
Step 3 — Prove the round-trip is lossless	29
Step 4 — Commit and protect	30
Step 5 — Baseline drift and you are done	30
Auditing content parity	30
Troubleshooting	31
Success checklist	31
Tutorial 06 — Bootstrap a New Workspace from One YAML File	32
The five probes	32
Step 1 — Find your capacity	33
Step 2 — Author workspace.yml	34
Step 3 — Dry-run: see what WOULD happen	34
Step 4 — Bootstrap for real	34
Step 5 — Prove convergence	35
Step 6 — Tear down (DESTRUCTIVE, scratch workspace only)	35
Where this goes next	35
Success checklist	35
Tutorial 07 — Roll Back a Release	37
The shape of the drill	37
Step 1 — Get a deployable source tree	37
Step 2 — Release A	38
Step 3 — Make the "bad" change and ship Release B	38
Step 4 — Ask the ledger what changed	38
Step 5 — Roll back to A	38
Step 6 — Verify the ledger tells the whole story	39
What to take into production	39
Success checklist	39
Tutorial 08 — Scheduled Drift Alerts to Teams	40
The moving parts	40
Step 0 — Pick the right workspace (read this twice)	42
Step 1 — Create the Teams incoming webhook	42
Step 2 — Store the webhook as a CI secret	42
Step 3 — Smoke-test the sink locally before involving CI	42
Step 4 — Trigger the workflow once by hand	43
Step 5 — Prove the alert path end-to-end	43
Step 6 — Let the schedule take over	43

Operational notes	43
Success checklist	44
Tutorial 09 — PR Review Bot for Semantic-Model and Lakehouse Diffs	45
The problem this solves	45
Step 1 — Diff locally with a synthetic change (zero credentials)	47
Step 2 — Understand what it watches	47
Step 3 — Post for real once	48
Step 4 — Wire it into CI	48
Step 5 — Prove the no-change behaviour	49
Troubleshooting	49
Exit codes (for CI wiring)	49
Success checklist	49
Tutorial 10 — Governance Sweeps: RBAC, Labels, Tenant Settings	50
Why these three together	50
Part A — RBAC audit	51
Part B — Sensitivity label sweep	52
Part C — Tenant-settings baseline	52
Success checklist	53
Tutorial 11 — Environments and Libraries	54
The mental model	54
Step 1 — Get a wheel to deploy	56
Step 2 — Sync the wheel into the Environment	56
Step 3 — Confirm a notebook actually imports it	57
What to take into production	57
Success checklist	57
Tutorial 12 — Config-Driven Auto-Update	58
Why a manifest, not a loop	58
Step 1 — Write environments.yml	60
Step 2 — Dry-run first, always	60
Step 3 — Run it for the non-gated targets	61
Step 4 — Production is a deliberate, separate act	61
Step 5 — Wire the automatic trigger (safely)	62
What to take into production	62
Success checklist	62

List of Figures

Figure 1: Tutorials — Learn Sigantry by Doing [LANDSCAPE](#)

Figure 2: Setup and First Contact [LANDSCAPE](#)

Figure 3: Sync Notebooks into a Workspace [LANDSCAPE](#)

Figure 4: Detect Drift

Figure 5: Read the Audit Trail

Figure 6: Adopt an Existing Workspace (Brownfield)

Figure 7: Bootstrap a New Workspace from One YAML File

Figure 8: Roll Back a Release

Figure 9: Scheduled Drift Alerts to Teams [LANDSCAPE](#)

Figure 10: PR Review Bot for Semantic-Model and Lakehouse Diffs [LANDSCAPE](#)

Figure 11: Governance Sweeps: RBAC, Labels, Tenant Settings

Figure 12: Environments and Libraries [LANDSCAPE](#)

Figure 13: Config-Driven Auto-Update

Tutorials — Learn Sigantry by Doing

Each tutorial is a worked, end-to-end example you can follow verbatim. Every command was executed (or, where destructive, dry-run-verified) against a live Fabric tenant on 2026-06-11; expected output blocks show what you should actually see.

If you are brand new, do tutorials 01 and 02 in order — everything else builds on them. After that, pick by need.

Learning path

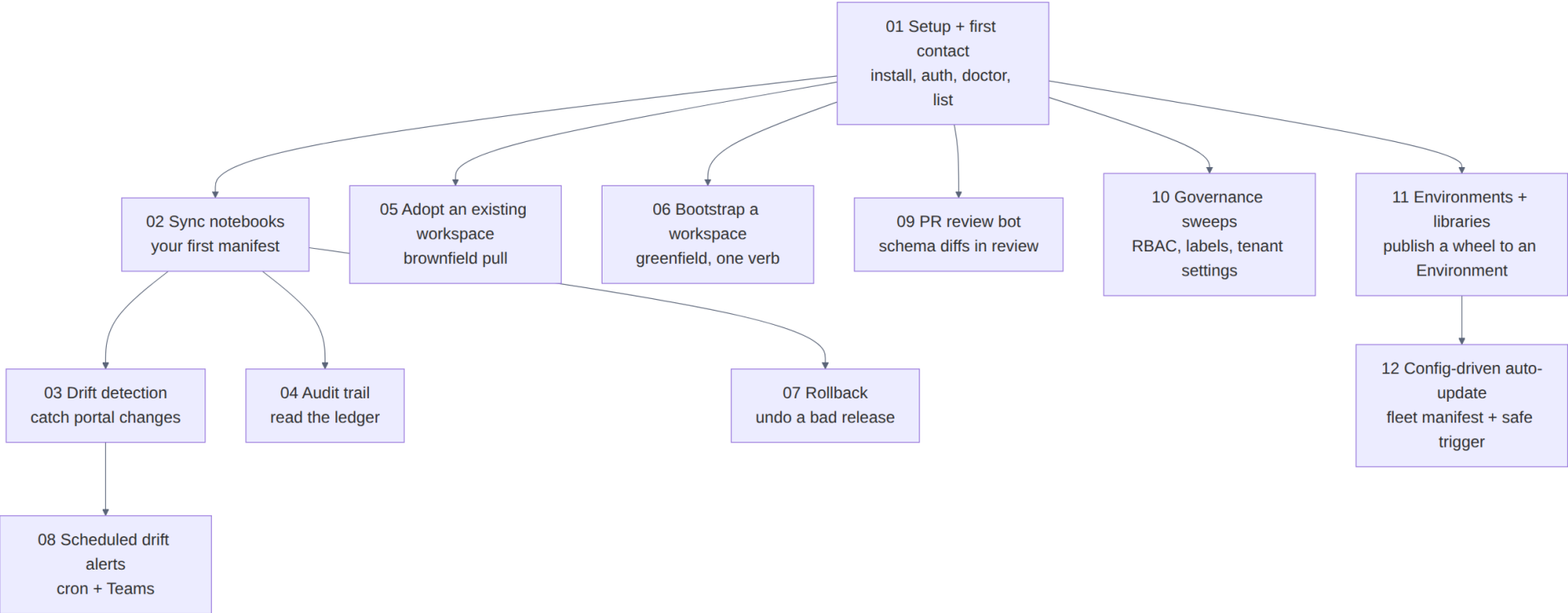


Figure 1: Tutorials — Learn Sigantry by Doing

The tutorials

#	Tutorial	You will achieve	Time
01	Setup and first contact	Working install, authenticated session, your workspaces listed and snapshotted	15 min
02	Sync notebooks into a workspace	Local notebooks governed by a manifest, published to Fabric, idempotency proven	30 min
03	Detect drift	A drift baseline; a deliberately-injected change detected and reconciled	20 min
04	Read the audit trail	Find, inspect, diff and hash-verify the release records your work created	15 min
05	Adopt an existing workspace	A live workspace pulled into a committed manifest, round-trip proven lossless	25 min
06	Bootstrap a new workspace	A workspace + capacity + folders + Git wiring from one YAML file	25 min
07	Roll back a release	Two releases in the ledger, a diff between them, the first restored	30 min
08	Scheduled drift alerts	A cron workflow that posts to Teams when a workspace drifts	30 min
09	PR review bot	Semantic-model and Lakehouse schema diffs posted on a pull request	25 min
10	Governance sweeps	RBAC export, sensitivity-label sweep, tenant-settings baseline	20 min
11	Environments and libraries	A custom wheel published to a Fabric Environment; you know exactly when notebooks see it	20 min
12	Config-driven auto-update	Many Environments kept current from one <code>environments.yml</code> ; a safe on-release auto-trigger	30 min

Conventions used in every tutorial

- **Two planes, two verbs.** `sync apply` and `sigantry diff` govern **topology** — an item's existence, name and folder. They never read or write the **content** of an item that already exists (`--with-publish` pushes content exactly once, at first publish). Content updates and republishes are the deploy engine's job: `sigantry deploy run` ([Tutorial 07](#)). A clean drift summary (`=N`) therefore certifies topology only — to verify content too, use the read-only recipe in "Auditing content parity" ([Tutorial 05](#)).

- `<workspace-guid>` etc. are placeholders — substitute your real values. Everything else is copy-paste literal.
- Expected output is shown after `# expect:` comments or in its own block. Minor differences (timestamps, GUIDs, counts) are normal; structural differences are not.
- Commands marked **DESTRUCTIVE** change or delete things and always require `--force`-style acknowledgement flags. Tutorials only point these at workspaces you created in the same tutorial.
- Each tutorial ends with a **Success checklist** — if every box ticks, you achieved the goal.

Prerequisites common to all tutorials

Covered once in [tutorial 01](#): a Python 3.11 environment with the toolkit installed, an Azure identity (`az login` or a service principal), and at least Contributor access to one Fabric workspace on a capacity. Service-principal and CI auth paths are summarised in tutorial 01's headless aside and fully specified in the User guide section 6.

Related reading

- Capability catalogue — the full verb inventory with source citations.
- Operator runbooks — terser operational references for each verb.
- Getting started — installation detail.
- User guide — the narrative operator manual.

Tutorial 01 — Setup and First Contact

Goal: a working Sigantry install, an authenticated session, and proof you can see your Fabric estate — workspaces listed, plugins discovered, one workspace snapshotted to JSON.

Time: ~15 minutes.

What you need before starting

1. **Python 3.11** via conda (first-class path) or venv (fallback).
2. **An Azure identity** that can reach your Fabric tenant. Easiest: the Azure CLI (`az login`). Headless/CI: a service principal with the "Service principals can call Fabric public APIs" tenant setting enabled.
3. **Viewer access or better** to at least one Fabric workspace.

How the pieces fit

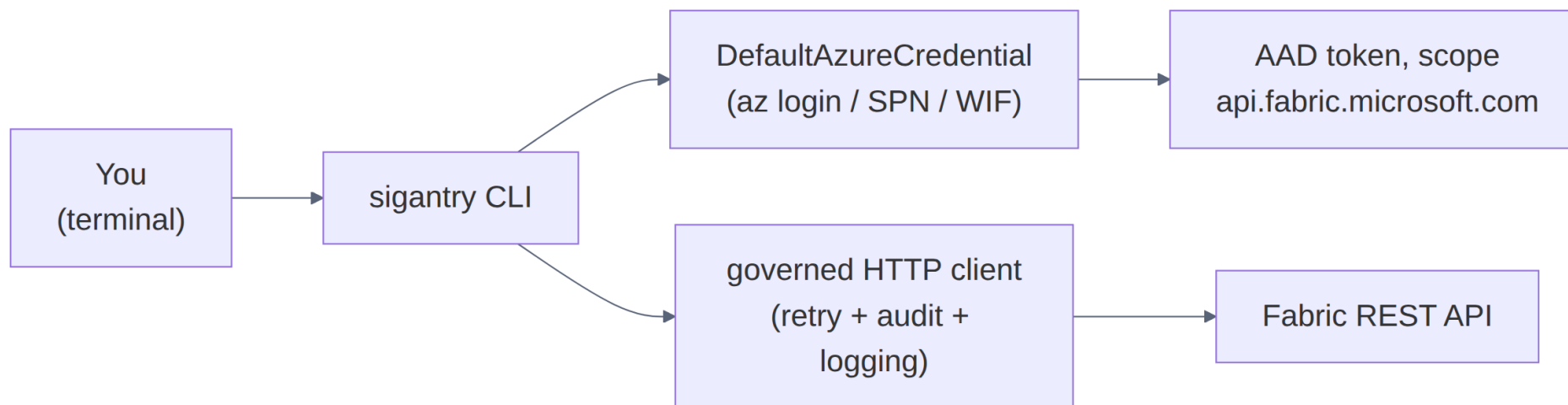


Figure 2: Setup and First Contact

Sigantry never stores credentials. Every command resolves a token through `DefaultAzureCredential`, which tries (in order): environment variables, workload identity, managed identity, and finally your `az login` session. For this tutorial, `az login` is all you need.

Step 1 — Install

```
git clone <your-monorepo-or-mirror-url> sigantry && cd sigantry
make conda-create                # creates the dedicated conda env
conda activate fabric-dataops-toolkits
make install-dev                 # editable install + dev extras
```

No conda? `python3.11 -m venv .venv && source .venv/bin/activate && pip install -e "[dev]"`.

Verify the entry point exists:

```
sigantry --help | head -5
# expect: a usage banner listing subcommands (workspace, sync, deploy, diff, release, ...)
```

Full installation detail, including offline installs: [getting-started/install.md](#).

Step 2 — Authenticate

```
az login                # browser opens; pick the account with Fabric access
az account show --query user.name -o tsv
# expect: your@email
```

Prove the token actually mints for the Fabric resource (this is the call that fails when your account lacks tenant access — better to find out now):

```
az account get-access-token --resource https://api.fabric.microsoft.com --query expiresOn -o tsv
# expect: a timestamp roughly 60-90 minutes in the future
```

Aside — running headless or as a service principal (optional)

The tutorials run fine on `az login` alone — skip this on first read. But every command in every tutorial also works unchanged under a service principal (SPN): the credential is ambient, resolved through `DefaultAzureCredential` at call time, never passed as a flag or config entry. Two prerequisites for any SPN: the tenant setting **"Service principals can call Fabric public APIs"** must be enabled, and the SPN needs a role on the target workspace.

For a headless shell without workload identity, set the three standard Azure variables before any verb (shell or CI secret store only — never an `.env` file; the project policy bans secret-bearing files):

```
export AZURE_TENANT_ID="<tenant-guid>"
export AZURE_CLIENT_ID="<spn-app-id>"
export AZURE_CLIENT_SECRET="<spn-secret>" # legacy mode -- prefer WIF in CI
```

In CI, prefer **workload identity federation (WIF)**: `azure/login` with only `client-id` + `tenant-id`, and the SPN proves itself with a short-lived OIDC token — no secret exists to store, rotate, or leak. Note that Key Vault is not the bootstrap path for the login credential (reading Key Vault already requires one); it holds *downstream* secrets via the SecretStore seam.

Verify whichever path you chose with the bundled auth doctor (a standalone script, not a `sigantry` sub-command):

```
diagnose-auth
# exit 0 = healthy; 2 = degraded (token works but e.g. the tenant toggle is
# off); 3 = no credential produced a token; --dry-run prints the plan offline
```

The full credential decision tree, all four supported auth paths and the per-verb scope checklist live in the User guide section 6.

Step 3 — Health check the install

```
sigantry doctor
# expect: a table of discovered plugins ending in a line like
# "N plugin(s) across M seam group(s)"
```

`doctor` enumerates every plugin registered against Sigantry's protocol seams. On a base install you will see the bundled reference implementations (notification sinks, secret stores, approval gates). A plugin whose import fails shows `Status = error` — that is your first place to look when something misbehaves later.

Step 4 — First contact: list your workspaces

```
sigantry workspace list
# expect: one row per workspace your identity can see, with id + display name
```

Every row's `id` is a GUID you will use constantly. Pick one workspace you have at least Contributor on and export it for the rest of the tutorials:

```
export WSID=<workspace-guid>
sigantry workspace get "$WSID"
# expect: JSON with displayName, type, capacityId
```

Step 5 — Snapshot it

```

sigantry sync snapshot --workspace-id "$WSID" --output /tmp/snap.json
python3 -c "import json; s=json.load(open('/tmp/snap.json')); print('folders:', len(s['folder_path_in
# expect: folders: <n> items: <m> -- matching what the Fabric portal shows

```

You will see a one-time **Preview-API warning** about the Fabric Folders REST endpoint. That is expected — folders are still a Preview API upstream. Acknowledge it permanently by setting `work-flow.preview_apis_acknowledged = true` in `.fabric-dataops.toml`.

Open the Fabric portal, count the folders and items in that workspace, and compare. They should match 1:1 — that is your proof the toolkit sees exactly what you see.

What just happened

You exercised the full stack once: credential resolution, the governed HTTP client (notice the structured JSON log lines with `elapsed_ms` and `retry_count`), and the read-only introspection surface. Nothing was written to the workspace; the only artefact is `/tmp/snap.json` on your machine.

Troubleshooting

Symptom	Cause	Fix
<code>sigantry: command not found</code>	env not activated	<code>conda activate fabric-dataops-toolkits</code>
<code>DefaultAzureCredential failed to retrieve a token</code>	no az session	<code>az login</code> , re-check <code>az account show</code>
<code>401</code> on every call	account in the wrong tenant	<code>az login --tenant <tenant-id></code>
<code>403</code> on a specific workspace	no role on that workspace	ask the workspace admin for Viewer+
doctor shows <code>Status = error</code> for a plugin	plugin import failed	the reason is in the Status column itself; add <code>--strict</code> to turn it into a non-zero exit for CI

Success checklist

- `[ ] sigantry --help` prints the command catalogue
- `[ ] az account get-access-token --resource https://api.fabric.microsoft.com` returns a future timestamp
- `[ ] sigantry doctor` lists plugins with Status `ok`

- [] `sigantry workspace list` shows your workspaces
- [] `/tmp/snap.json` folder + item counts match the portal

Next: [Tutorial 02 — Sync notebooks into a workspace.](#)

Tutorial 02 — Sync Notebooks into a Workspace

Goal: a folder of local `.ipynb` notebooks governed by a `sync.yml` manifest: declared once, published into the right workspace folder, with re-runs proven to be safe no-ops. This is Sigantry's flagship workflow.

Time: ~30 minutes. **Builds on:** [Tutorial 01](#) (`$WSID` exported, auth working).

The mental model

A **manifest** declares the desired topology: which local files become which workspace items, in which folders. `sync apply` converges the workspace toward the manifest — additively by default (it creates folders and moves items; it never deletes unless you explicitly opt in). Items that do not exist yet need `--with-publish` to be created the first time.

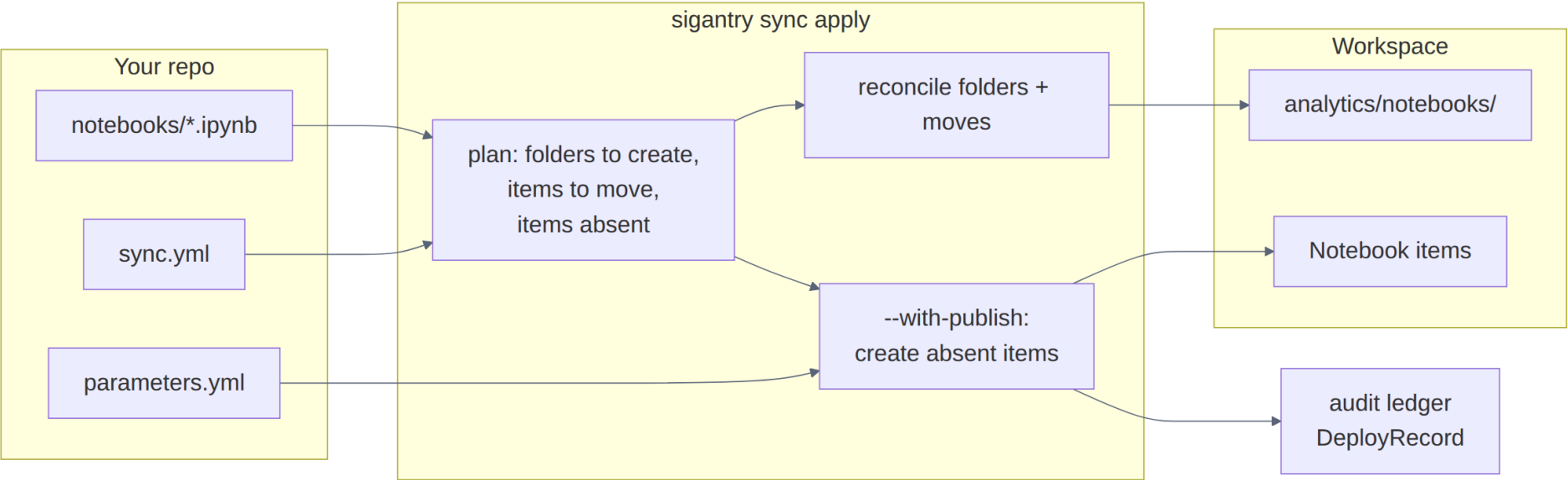


Figure 3: Sync Notebooks into a Workspace

Step 1 — Make a working folder with two notebooks

Use real notebooks if you have them; otherwise create minimal valid ones:

```
mkdir -p ~/sigantry-tut02 && cd ~/sigantry-tut02
python3 - <<'PY'
import json
nb = {"cells": [{"cell_type": "code", "source": ["print('hello fabric')"],
              "metadata": {}, "outputs": [], "execution_count": None},
          "metadata": {}, "nbformat": 4, "nbformat_minor": 5}
for name in ("ingest_orders.ipynb", "transform_orders.ipynb"):
    json.dump(nb, open(name, "w"), indent=1)
print("wrote 2 notebooks")
PY
```

Step 2 — Author the manifest

Create `sync.yml` next to the notebooks. `local_path` is relative to the manifest file; `target_folder` is the workspace folder path (created if missing); `display_name` is what the portal shows.

```
schema_version: "1.0.0"
items:
  - {local_path: ingest_orders.ipynb, type: Notebook, target_folder: analytics/notebooks, display_
  - {local_path: transform_orders.ipynb, type: Notebook, target_folder: analytics/notebooks, display_
folders: []
```

The top-level `folders:` list is a **preservation set** — folders you list there are never deleted even under orphan cleanup. Leave it empty for now.

Field-by-field reference: [sync-schema.md](#).

Step 3 — Author a minimal parameters.yml

First-time publish hands content to the deploy engine, which requires a parameters file even when there is nothing to substitute:

```
# parameters.yml -- minimal valid file; no replacements needed yet
find_replace: []
```

Validate it (catches hard-coded GUIDs and unset `$ENV:` references early):

```
sigantry config validate parameters.yml
# expect: OK -- 0 environment(s) parsed:
```

Step 4 — Dry-run

Always preview first. Dry-run computes the plan and exits without touching Fabric:

```
sigantry sync apply --manifest sync.yml --workspace-id "$WSID" --dry-run
# expect (fresh workspace):
#   dry-run would create 2 folder(s) and move 0 item(s) for 2 manifest item(s).
# (2 folders because nested paths create parent-first: analytics, then
# analytics/notebooks. If analytics already exists you will see 1.)
```

Read the plan critically: folder creates and item moves are shown; items that do not exist yet are NOT in this plan (they surface in the publish step next). If the plan shows moves you did not expect, your `target_folder` paths disagree with reality — stop and check before applying.

Step 5 — Apply with first-time publish

```
sigantry sync apply --manifest sync.yml --workspace-id "$WSID" \
  --with-publish --params parameters.yml
# expect, per absent item:
#   Publishing Notebook 'ingest_orders'
#   Published Notebook 'ingest_orders'
# and finally:
#   sync apply succeeded release_id=sync-publish-<timestamp>
#   items_packaged=2 folders_created=2 items_moved=0
```

Note the `release_id` — that is your audit-ledger key (Tutorial 04 uses it).

Open the Fabric portal: the workspace now has `analytics/notebooks/` containing both notebooks. This portal check is the one step a machine cannot do for you.

Step 6 — Prove idempotency

The property that makes this safe to automate: running the same command again changes nothing.

```
sigantry sync apply --manifest sync.yml --workspace-id "$WSID" \
  --with-publish --params parameters.yml
# expect: sync apply succeeded ... folders_created=0 items_moved=0
# and NO "Publishing ..." lines -- the absent set is empty, publish short-circuits
```

What just happened — and what deliberately did NOT

- The manifest now governs those two notebooks: their existence, names and folder.
- **Content of already-existing items is not re-pushed by sync.** The sync layer is topology-only by design; pushing changed cell content into an existing notebook is the deploy engine's job (`sigantry deploy run` — see [Tutorial 07](#) for that surface). This boundary is why re-runs are safe. To prove

workspace content matches your local files, use the read-only pull-and-compare recipe in "Auditing content parity" ([Tutorial 05](#)).

- Two `DeployRecord` entries (one per apply) landed in `~/.sigantry/audit/deploy.jsonl`.

Troubleshooting

Symptom	Cause	Fix
<code>--with-publish</code> requires <code>--params <parameters.yml></code>	forgot <code>--params</code>	add <code>--params parameters.yml</code>
<code>HardcodedGuidError</code> on validate	raw GUID in parameters.yml	use <code>\$items. / \$workspace. / \$ENV:</code> forms
publish succeeds but item lands at workspace root	<code>target_folder</code> typo	fix the path; re-run apply (it will move the item)
<code>403</code> on apply	Viewer-only role	you need Contributor+ to create folders/items
second apply still shows <code>Publishing ...</code>	<code>display_name</code> in manifest differs from the published item	make <code>display_name</code> match exactly; the absent-set join is on (<code>display_name</code> , <code>type</code>)

Success checklist

- `[] config validate` exits 0
- `[]` dry-run plan matched your expectation before you applied
- `[]` portal shows both notebooks inside `analytics/notebooks/`
- `[]` re-run printed `folders_created=0 items_moved=0` with no publish lines
- `[] sigantry release list | head -4` shows two `sync-publish-...` releases

Next: [Tutorial 03 — Detect drift](#), using this same manifest.

Tutorial 03 — Detect Drift

Goal: establish a drift baseline for the manifest from Tutorial 02, deliberately break it from the portal, watch `sigantry diff` catch the change, then reconcile.

Time: ~20 minutes. **Builds on:** [Tutorial 02](#) (manifest applied, `$WSID` exported, working dir `~/sigantry-tut02`).

What drift means here

Drift = the live workspace disagrees with your committed manifest. Someone renames a notebook in the portal, drags it to another folder, or deletes it — and two weeks later your repo and your workspace silently describe different worlds. `sigantry diff` compares the manifest against a live snapshot and classifies every item:

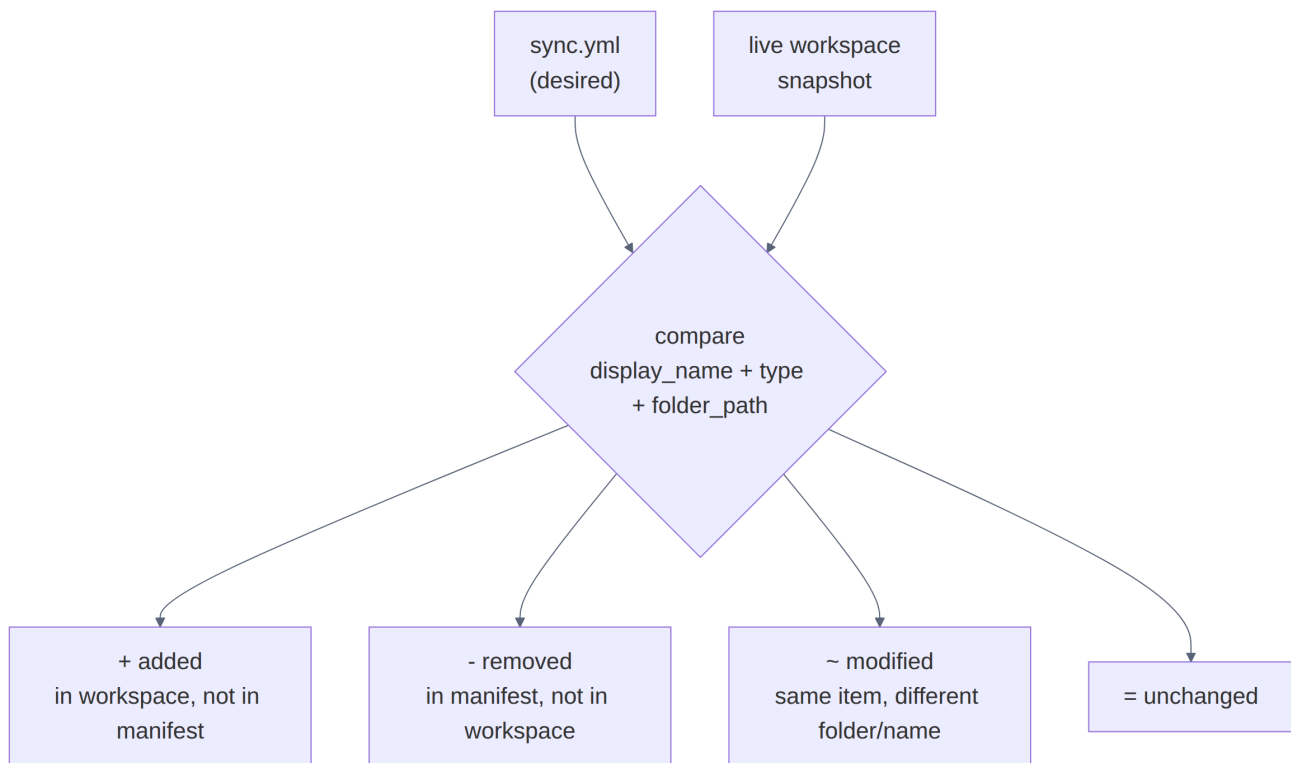


Figure 4: Detect Drift

Comparison is **metadata-only by design** (name, type, folder). Cell-content changes inside a notebook are not drift in this sense — content flows through Git integration or the deploy engine, not the sync layer. A clean `=N` therefore certifies topology only; for the content side, see "Auditing content parity" in [Tutorial 05](#).

Step 1 — Baseline: prove you currently have no drift

```
cd ~/sigantry-tut02
sigantry diff --manifest sync.yml --workspace-id "$WSID"
# expect a table whose rows for your 2 items show "=" (unchanged), ending:
#   drift summary +<n> -0 ~0 =2
```

Read the summary line carefully — this is the most misread output in the toolkit:

- `=2` — both manifest items match the workspace. That is YOUR signal.
- `+<n>` — items that exist in the workspace but not in your manifest. In a workspace you share with other teams (or that has anything beyond your two notebooks), `+` counts everything else that lives there. **`+` is only meaningful when the manifest is intended to govern the entire workspace.**
- `-` and `~` are the alarming ones: something you declared is missing or moved.

Step 2 — Inject drift from the portal

In the Fabric portal: open the workspace, find `transform_orders`, and **rename it** to `transform_orders_TEMP` (or drag it out of `analytics/notebooks/` — either works).

This simulates the classic incident: a well-meaning colleague "just quickly fixes" something in the UI.

Step 3 — Catch it

```
sigantry diff --manifest sync.yml --workspace-id "$WSID"
# expect: transform_orders now shows as "-" (removed) and the renamed item as "+"
#   drift summary +<n+1> -1 ~0 =1
```

A rename is reported as remove+add because identity is matched on (`display_name`, `type`). A pure folder move of an unrenamed item shows as `~ modified`.

For CI, get the same answer machine-readably and as an exit code:

```
sigantry diff --manifest sync.yml --workspace-id "$WSID" --output json --no-hint > /tmp/drift.json
sigantry diff --manifest sync.yml --workspace-id "$WSID" --fail-on-drift --no-hint; echo "exit=$?"
# expect: exit=1 (clean would be exit=0)
```

`/tmp/drift.json` follows a SemVer-pinned schema (`docs/reference/drift-schema.json`) — safe to parse in pipelines.

Step 4 — Reconcile

Two valid resolutions, depending on which side is right:

The manifest is right (usual case) — undo the portal change. Rename the notebook back in the portal, or if the drift was a folder move, `sync apply` will move it home:

```
sigantry sync apply --manifest sync.yml --workspace-id "$WSID"
```

Note: `sync apply` re-converges folder placement for declared items, but it cannot un-rename an item (a renamed item no longer matches the manifest identity). Renames get fixed in the portal, or by deleting the renamed item and re-publishing.

The workspace is right — the change was intentional. Update `sync.yml` to declare the new name, commit it, and the next diff is clean. The manifest is the contract; changing the contract is fine, bypassing it is not.

Step 5 — Confirm the clean state

```
sigantry diff --manifest sync.yml --workspace-id "$WSID" --fail-on-drift --no-hint; echo "exit=$?"  
# expect: exit=0 and drift summary ... -0 ~0 =2
```

Caveat that will save you a bad on-call page

Do not wire `--fail-on-drift` against a **shared workspace** your manifest only partially governs: every other team's item counts as `+ added` and the check is permanently red. Scope scheduled drift checks (Tutorial 08) to workspaces the manifest fully owns, or treat `-/~` as the alert condition when consuming the JSON.

Success checklist

- [] baseline run showed `=2` with `-0 ~0`
- [] after the portal rename, diff reported `-1` and exit code 1 with `--fail-on-drift`
- [] after reconciling, diff is back to `-0 ~0 =2` and exit code 0
- [] you can explain to a colleague why `+154` might be fine but `-1` never is

Next: [Tutorial 04 — Read the audit trail](#) — every apply you just ran left evidence; go look at it.

Tutorial 04 — Read the Audit Trail

Goal: find the release records your previous tutorials created, inspect one, diff two of them, and verify a record's tamper-evident hash — including watching the verification fail when you tamper with a copy.

Time: ~15 minutes. **Builds on:** [Tutorial 02](#) (you ran at least two applies).

Where the evidence lives

Sigantry maintains append-only JSONL ledgers under `~/.sigantry/audit/`. Every record carries an `audit_hash` — a SHA-256 over the canonical JSON of all its other fields — so any post-hoc edit is detectable.

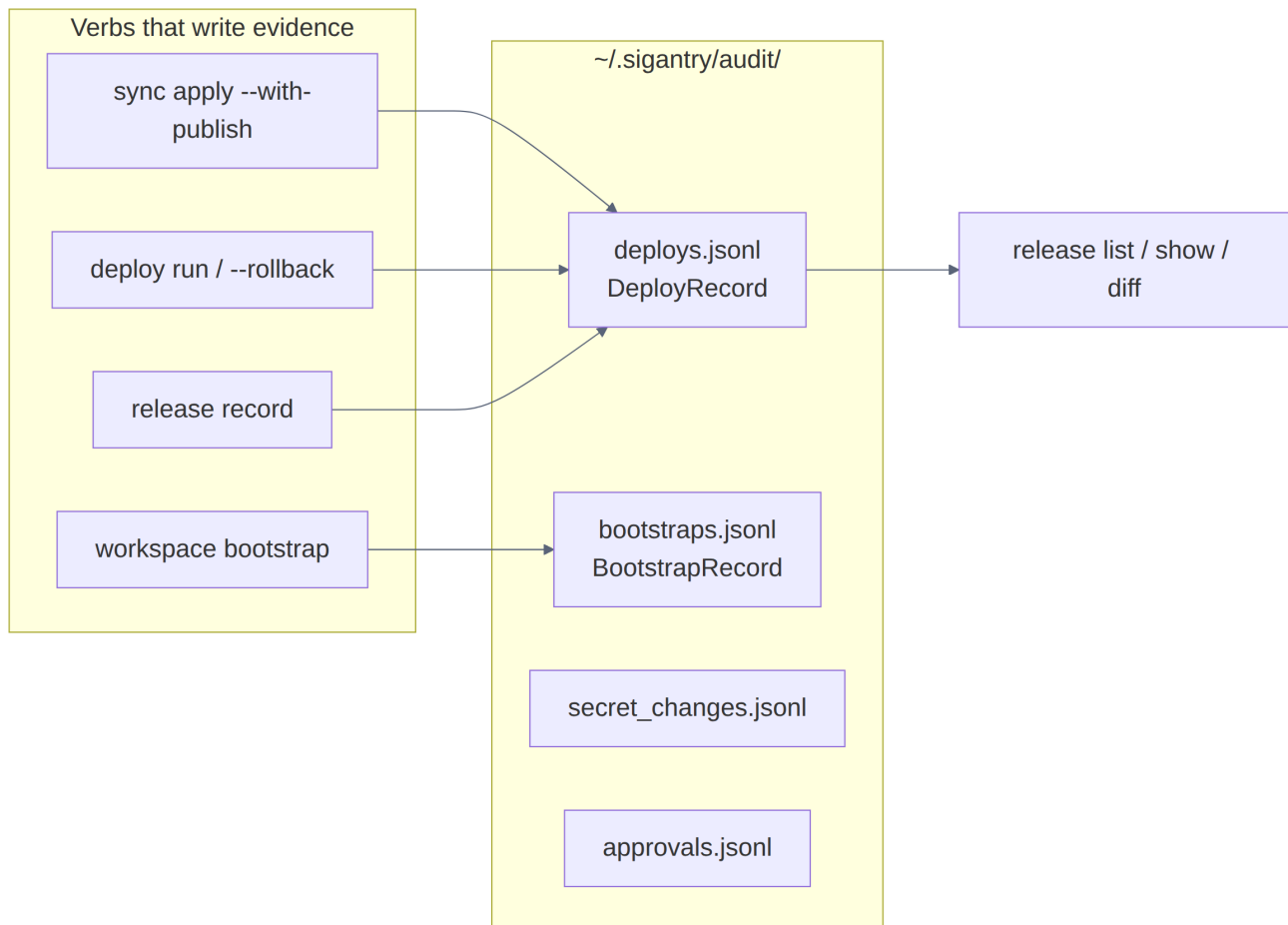


Figure 5: Read the Audit Trail

Step 1 — List your releases

```
sigantry release list --limit 5
# expect: a table, most recent first, including your two
# sync-publish-<timestamp> rows from Tutorial 02
```

Filter to one workspace (substring match on the workspace field):

```
sigantry release list --workspace "$WSID" --json | python3 -m json.tool | head -20
```

Step 2 — Inspect one record

Take a `release_id` from the list:

```
sigantry release show sync-publish-<timestamp>
# expect: pretty-printed JSON with release_id, workspace, fabric_items_changed[],
# test_evidence{}, approver, created_at, audit_hash
```

What the fields mean operationally: `fabric_items_changed` is the item set this release touched; `test_evidence` is whatever the emitting verb captured (for sync-publish: the moved/published item lists); `audit_hash` is the integrity seal.

Step 3 — Diff two releases

```
sigantry release diff <earlier-release-id> <later-release-id> --json
# expect JSON with "added" / "removed" / "unchanged" arrays of
# {logical_name, item_type, fabric_item_id}
```

This answers the release-manager question — "what changed between Tuesday's and Thursday's deploy?" — from the ledger alone, without touching Fabric.

Step 4 — Verify integrity, then break it on purpose

The hash check is exposed on the record class. Verify the newest record:

```
python3 - <<'PY'
import json, pathlib
from sigantry_core.release.record import DeployRecord
path = pathlib.Path.home() / ".sigantry/audit/deploy.jsonl"
rec = DeployRecord(**json.loads(path.read_text().splitlines()[-1]))
print("release:", rec.release_id)
print("verify_hash():", rec.verify_hash())
PY
# expect: verify_hash(): True
```

Now prove the seal means something — tamper with a COPY and watch it fail:

```
python3 - <<'PY'
import json, pathlib
from sigantry_core.release.record import DeployRecord
path = pathlib.Path.home() / ".sigantry/audit/deploy.jsonl"
raw = json.loads(path.read_text().splitlines()[-1])
raw["approver"] = "mallory@example.com"          # the tamper
print("tampered verify_hash():", DeployRecord(**raw).verify_hash())
PY
# expect: tampered verify_hash(): False
```

One changed byte anywhere in the record flips verification to `False`. This is what "verify without trust" means: an auditor does not have to trust the file, the operator, or the machine — only the algorithm.

Step 5 — (Optional) link a release to a work item

If you track work in GitHub issues or Azure DevOps work items, `release record` writes a structured comment back onto the item so the ticket shows what shipped:

```
export GITHUB_TOKEN=<a-pat-with-repo-scope>
sigantry release record --provider github \
  --release-id manual-$(date -u +%Y%m%dT%H%M%SZ) \
  --workspace "$WSID" \
  --work-items 42 \
  --approver you@example.com \
  --fabric-items "ingest_orders.Notebook,transform_orders.Notebook" \
  --test-evidence '{"smoke":"passed"}' \
  --github-owner <owner> --github-repo <repo>
# expect: one-line summary; issue #42 gains a release comment with the audit hash
```

ADO works the same with `--provider ado --ado-organization ... --ado-project ...`.

A note on ledger hygiene

The ledger is per-machine (`~/.sigantry/audit/`). For CI, pass `--audit-dir` to give each pipeline run a hermetic ledger, and archive the directory as a build artefact — that turns every pipeline run into auditable evidence.

Success checklist

- `release list` shows your Tutorial 02 releases
- `release show` displays a full record with an `audit_hash`
- `release diff` between two releases returns the added/removed/unchanged sets
- untampered record verifies `True`; tampered copy verifies `False`

Next: [Tutorial 05 — Adopt an existing workspace](#) or jump to [Tutorial 07 — Roll back a release](#).

Tutorial 05 — Adopt an Existing Workspace (Brownfield)

Goal: take a workspace that was built by hand in the portal and bring it under manifest control — pulled into a committed `sync.yml` + sources tree, with the round-trip proven lossless (re-applying the pulled manifest is a no-op).

Time: ~25 minutes. **Builds on:** [Tutorial 01](#). Works on any workspace you have Contributor on — including one full of items you did not create.

Why this matters

Most real estates are brownfield: years of hand-built notebooks, pipelines and folders. You cannot govern what is not declared. `sync pull` is the adoption verb: it reads the live workspace and emits the manifest that *describes what already exists* — without disturbing anything (pull is strictly read-only).

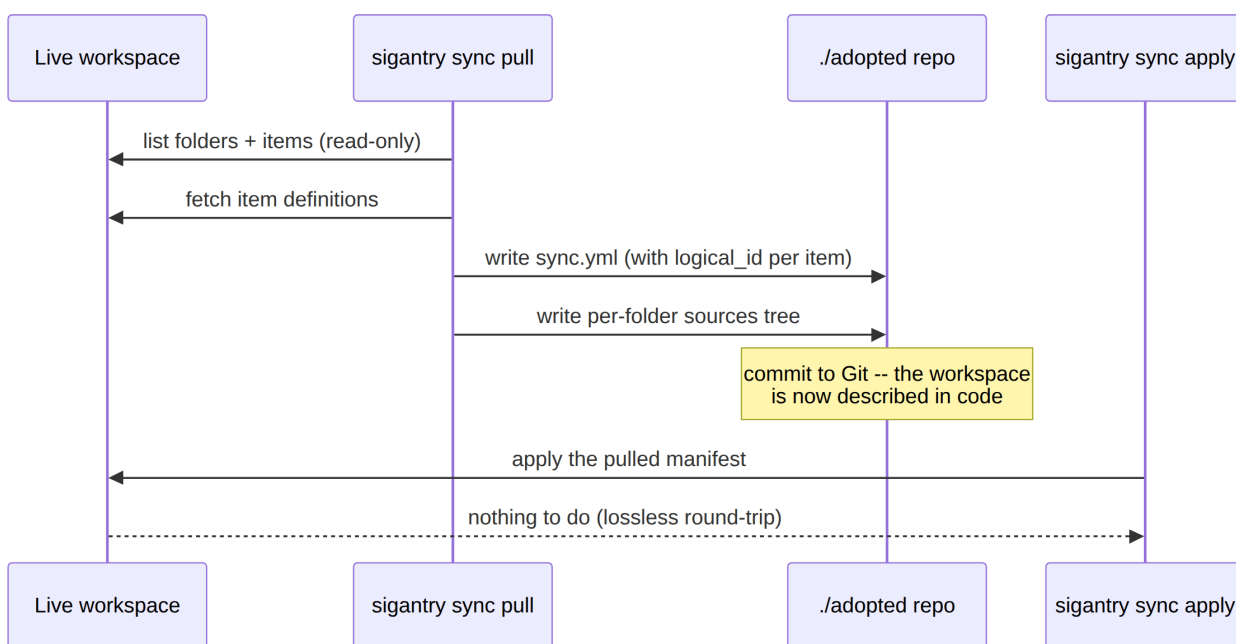


Figure 6: Adopt an Existing Workspace (Brownfield)

The key detail is `logical_id`: pull preserves each item's workspace identity in the manifest, so a later apply recognises every item as "already there" instead of trying to recreate it.

Step 1 — Pull

```
mkdir -p ~/sigantry-tut05 && cd ~/sigantry-tut05
sigantry sync pull --workspace-id "$WSID" --into ./adopted
# expect: sync pull succeeded items_pulled=<n> sync_yaml=./adopted/sync.yml
```

The target directory must be empty (or absent); pass `--force` only when you knowingly want to overwrite a previous pull.

By default pull takes the common item types (Notebook, DataPipeline, SemanticModel, Report, SparkJobDefinition). Narrow it with `--type`:

```
sigantry sync pull --workspace-id "$WSID" --into ./adopted-notebooks --type Notebook
```

Step 2 — Read what you got

```
head -25 ./adopted/sync.yml
# expect: schema_version, then one items[] entry per pulled item, each carrying
#   local_path, type, target_folder, display_name and logical_id
ls ./adopted/
# expect: sync.yml plus one directory per top-level workspace folder; item
#   sources live at <folder-path>/<display_name>/ (there is no separate
#   sources/ directory -- the tree mirrors the workspace folder layout)
```

Inspect critically before committing:

- Do the `target_folder` paths mirror the portal's folder tree? They should, exactly.
- Every item has a `logical_id` GUID — that is the round-trip anchor. Do not edit these.
- Items of types pull does not handle were skipped, not broken — compare counts against your Tutorial 01 snapshot if you want the exact delta.

Step 3 — Prove the round-trip is lossless

Apply the manifest you just pulled, in dry-run first, then for real:

```
sigantry sync apply --manifest ./adopted/sync.yml --workspace-id "$WSID" --dry-run
# expect: dry-run would create 0 folder(s) and move 0 item(s) for <n> manifest item(s).

sigantry sync apply --manifest ./adopted/sync.yml --workspace-id "$WSID"
# expect: sync apply succeeded ... folders_created=0 items_moved=0
```

`0 / 0` is the proof: the manifest fully and faithfully describes the workspace. If the plan is NOT empty, the pull and the workspace diverged in the seconds between the two commands (someone is editing live), or you edited the manifest — investigate before proceeding.

Step 4 — Commit and protect

```
cd ./adopted && git init -q && git add -A && git commit -qm "adopt workspace $WSID under manifest con
```

If operators legitimately create scratch folders in this workspace via the UI, add those paths to the manifest's top-level `folders:` preservation list now — that is what keeps a future orphan-cleanup from eating them:

```
folders:
  - /scratch/UI-experiments
```

Step 5 — Baseline drift and you are done

```
sigantry diff --manifest ./adopted/sync.yml --workspace-id "$WSID"
# expect: every pulled item "=", summary -0 ~0 =<n>
```

From here the workspace is governed: Tutorial 03's drift loop and Tutorial 08's scheduled checks apply to it unchanged.

Auditing content parity

Drift detection certifies topology only — a clean `=N` says nothing about whether the *content* of workspace items still matches your local files. Because pull fetches full item definitions, it doubles as the read-only answer to that question: pull into a scratch directory and compare cell sources.

```
sigantry sync pull --workspace-id "$WSID" --into /tmp/content-audit --type Notebook
python3 - <<'PY'
import json, pathlib

def cells(path):
    nb = json.loads(pathlib.Path(path).read_text())
    return [(c["cell_type"], "".join(c.get("source", []))) for c in nb["cells"]]

local = pathlib.Path("<your-repo>/<notebook-name>.ipynb")
pulled = next(pathlib.Path("/tmp/content-audit").rglob("<notebook-name>/notebook-content.ipynb"))
print("content identical:", cells(local) == cells(pulled))
PY
# expect: content identical: True    (False = the two sides have diverged)
```

Compare cell sources, not raw bytes — Fabric strips outputs and rewrites notebook metadata, so byte-level diffs always differ even when the code is identical. If content has diverged and local is the truth, republish those items through the deploy engine (`sigantry deploy run`, [Tutorial 07](#)); if the workspace is the truth, re-pull and commit.

Troubleshooting

Symptom	Cause	Fix
<code>PullTargetNotEmptyError</code>	<code>--into</code> dir has files	use an empty dir, or <code>--force</code> deliberately
pulled count < portal count	unsupported item types skipped	check <code>--type</code> coverage; unsupported types stay portal-managed
round-trip apply shows moves	workspace changed mid-tutorial, or manifest edited	re-pull into a fresh dir; diff the two manifests
<code>403</code> on pull	Viewer role	pull fetches item definitions; you need Contributor+

Success checklist

- `[] sync pull` reported `items_pulled=<n>` and wrote `sync.yml` + the per-folder sources tree
- `[]` every manifest item carries a `logical_id`
- `[]` round-trip apply printed `folders_created=0 items_moved=0`
- `[]` the manifest is committed to Git
- `[] sigantry diff` shows `=<n>` with `-0 ~0`

Next: [Tutorial 06 — Bootstrap a new workspace](#) for the greenfield mirror image of what you just did.

Tutorial 06 — Bootstrap a New Workspace from One YAML File

Goal: a brand-new workspace materialised from a single `workspace.yml` — created, bound to a capacity, folder blueprint laid out — with a `BootstrapRecord` in the audit ledger and the re-run proven to converge to "nothing to do". Then torn down cleanly.

Time: ~25 minutes. **Builds on:** [Tutorial 01](#). You also need a **capacity GUID** you may create workspaces on (`sigantry capacity list` shows the capacities your identity can see; a Trial capacity works fine).

The five probes

`workspace bootstrap` runs a probe-before-act sequence: every step first checks current state and does nothing if that step is already satisfied. That is the property that makes it safe to keep in CI.

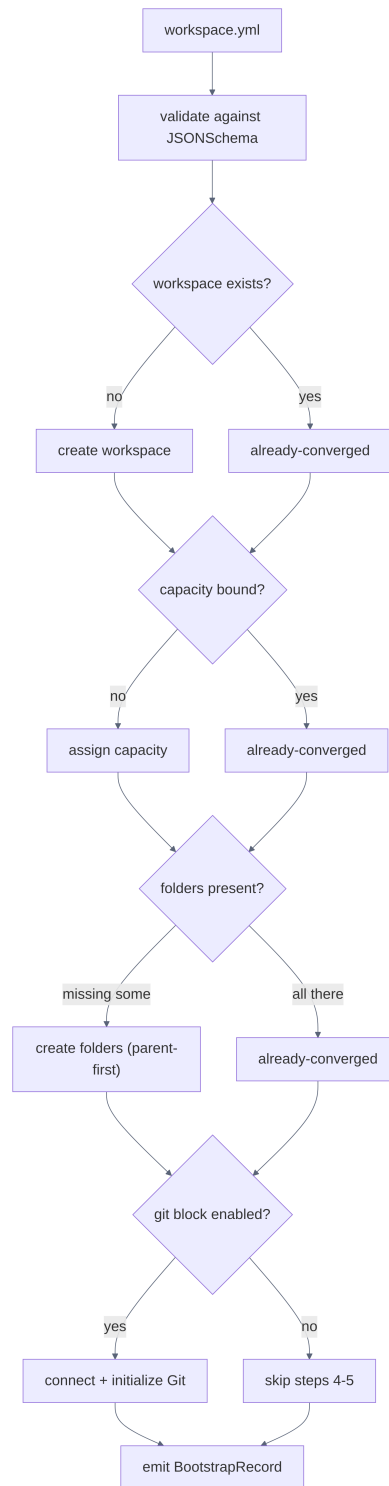


Figure 7: Bootstrap a New Workspace from One YAML File

Step 1 — Find your capacity

```
sigantry capacity list  
# expect: rows with capacity id + sku; copy the GUID of one you can use  
export CAPACITY_ID=<capacity-guid>
```

Step 2 — Author workspace.yml

The minimum viable manifest (no Git wiring — add that later once a repo exists):

```
schema_version: "1.0"
workspace:
  name: tut06-bootstrap-demo
  description: "Tutorial 06 scratch workspace"
  capacity_id: "<capacity-guid>" # paste $CAPACITY_ID; quotes required
  folders:
    blueprint: minimal_starter
  git:
    enabled: false
```

Two folder strategies exist: `blueprint:` picks a named layout from the catalogue (`minimal_starter` = 8 folders in numbered pipeline-flow order, 000 Orchestrate through 500 Visualize plus 999 Libraries and Archive; `medallion` is an alias for the same layout), or `list:` declares explicit folder names. Full schema with every key, including Git wiring and stage markers: workspace-bootstrap-operator.md.

Step 3 — Dry-run: see what WOULD happen

```
sigantry workspace bootstrap workspace.yml --dry-run
# expect: a JSON report ending like
#   "step_outcomes": {
#     "workspace": "created", "capacity": "created", "folders": "created",
#     "git": "skipped", "initialize": "skipped"
#   },
#   "audit_hash": "<sha-256>",
#   "dry_run": true
# "created" inside a dry_run:true envelope means WOULD-create. Nothing was POSTed.
```

A schema mistake exits 2 with a readable message pointing at the offending key — fix and re-run until the dry-run plan looks right.

Step 4 — Bootstrap for real

```
sigantry workspace bootstrap workspace.yml --operator you@example.com
# expect: the same JSON report shape as the dry-run, now with a real
#   "workspace_id", "step_outcomes" showing "created" for
#   workspace/capacity/folders ("skipped" for git/initialize),
#   an "audit_hash", and "dry_run": false
```

Check the portal: the workspace exists with the blueprint folder tree. Export the `workspace_id` straight from the JSON report:

```
export TUT06_WSID=<workspace_id-from-the-report>
```

Step 5 — Prove convergence

```
sigantry workspace bootstrap workspace.yml --operator you@example.com
# expect: workspace/capacity/folders report already-converged (git/initialize
#         stay skipped while git.enabled is false); a second BootstrapRecord
#         is written (the audit trail records the check itself)
```

This is the load-bearing property: bootstrap in CI cannot double-create, cannot fight a human who already made the workspace, and documents every convergence check it performs.

Verify the ledger:

```
python3 - <<'PY'
import json, pathlib
from sigantry_core.workspace.records import BootstrapRecord
path = pathlib.Path.home() / ".sigantry/audit/bootstraps.jsonl"
rec = BootstrapRecord(**json.loads(path.read_text().splitlines()[-1]))
print("step_outcomes:", rec.step_outcomes)
print("verify_hash():", rec.verify_hash())
PY
# expect: already-converged for workspace/capacity/folders, skipped for
#         git/initialize; verify_hash(): True
```

Step 6 — Tear down (DESTRUCTIVE, scratch workspace only)

Delete the tutorial workspace. Destructive verbs demand explicit acknowledgement and record who/why:

```
sigantry workspace delete "$TUT06_WSID" --force
# expect: 200; the deletion is audited
```

If the delete intermittently fails with Fabric's known `UnknownError`, the Python API exposes a Power-BI-API fallback (`delete_workspace(..., pbi_fallback=True)`); simply retrying the CLI delete usually also succeeds.

Where this goes next

In real use, `workspace.yml` lives in the consumer repo next to `sync.yml`: bootstrap makes the empty governed shell, Tutorial 02's sync-publish fills it, and the Git block (set `git.enabled: true` with your repo coordinates) wires source control from day one. For fleet-scale, state-managed provisioning instead, see the Terraform comparison in `CONSUMING.md`.

Success checklist

- [] dry-run reported the would-create plan without touching the tenant
- [] portal shows the workspace with the blueprint folders

- [] second run reported `already-converged` on every executed step
- [] `BootstrapRecord.verify_hash()` returns `True`
- [] scratch workspace deleted

Next: [Tutorial 07 — Roll back a release.](#)

Tutorial 07 — Roll Back a Release

Goal: two deliberate releases of an item tree into a scratch workspace, a ledger diff showing exactly what changed between them, then the first release restored with one audited command. After this drill, "bad deploy" stops being an incident and becomes a procedure.

Time: ~30 minutes. **Builds on:** [Tutorial 06](#) — use a scratch workspace (rollback supplants live state; never drill on a shared workspace). **Uses:** `sigantry deploy run` — the content-level deploy engine (distinct from the topology-level `sync apply` you used in Tutorial 02). This is also the verb for republishing the content of items `sync` already governs.

The shape of the drill

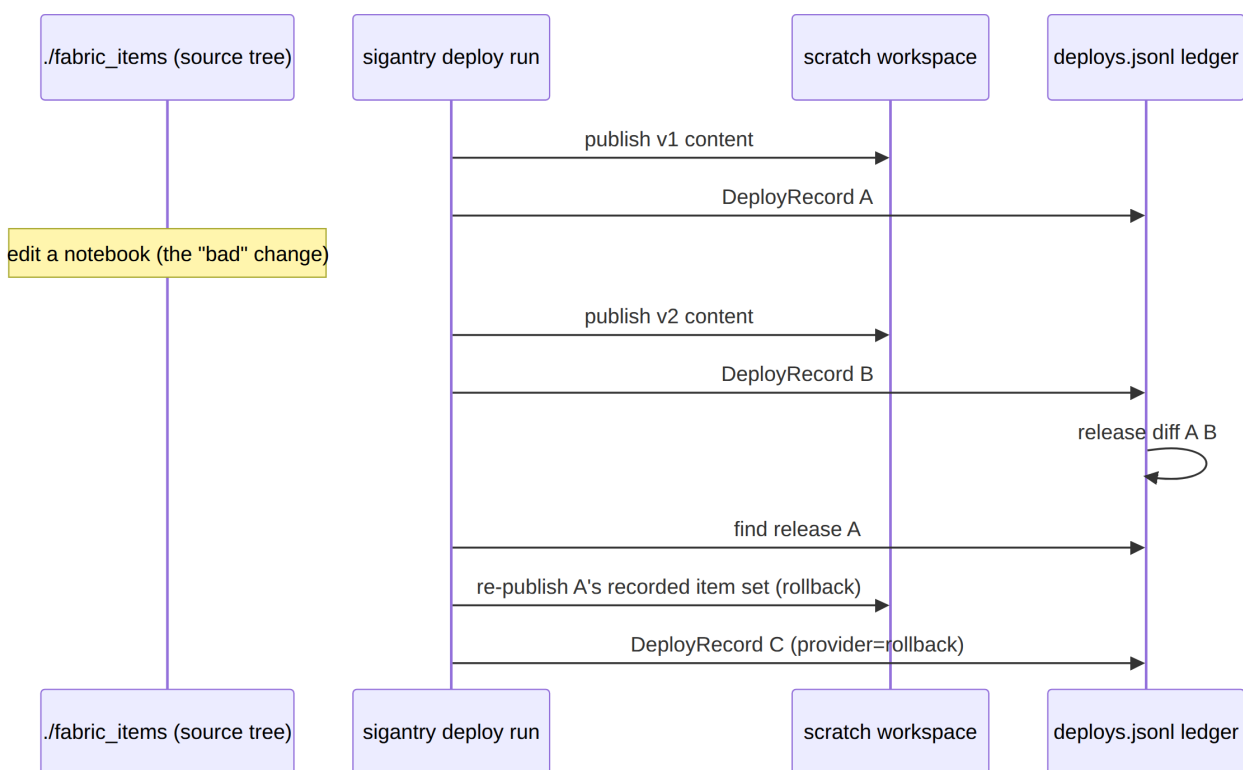


Figure 8: Roll Back a Release

Step 1 — Get a deployable source tree

`deploy run` consumes a Fabric **item tree** — directories like `<name>.Notebook/` each containing a `.platform` file plus content. The repo's demo scaffold ships a ready-made one; copy it so you can edit freely:

```
cp -r templates/demo/fabric_items ~/sigantry-tut07 && cd ~/sigantry-tut07
ls
# expect: one or more *.Notebook / *.DataPipeline directories
printf 'find_replace: []\n' > parameters.yml
```

(Any item tree exported by Fabric Git integration works the same way.)

Step 2 — Release A

Use a hermetic ledger dir for the drill so your real ledger stays clean:

```
export AUDIT_DIR=$(mktemp -d)
sigantry deploy run --source . --workspace-id "$TUT06_WSID" --environment DEV \
  --params parameters.yml --audit-dir "$AUDIT_DIR"
# expect: per-item "Published ..." lines, then a DeployRecord summary with a release_id
sigantry release list --audit-dir "$AUDIT_DIR"
# note the first release_id -> call it A
```

Step 3 — Make the "bad" change and ship Release B

Edit any notebook in the tree (add a cell, change a line — content is what deploy pushes), then deploy again:

```
sigantry deploy run --source . --workspace-id "$TUT06_WSID" --environment DEV \
  --params parameters.yml --audit-dir "$AUDIT_DIR"
sigantry release list --audit-dir "$AUDIT_DIR"
# expect: two records now; the newer is B
```

Step 4 — Ask the ledger what changed

```
sigantry release diff <release-A-id> <release-B-id> --audit-dir "$AUDIT_DIR" --json
# expect: added/removed/unchanged item sets between the two releases
```

In a real incident this is your blast-radius answer before you touch anything.

Step 5 — Roll back to A

Rollback is **DESTRUCTIVE by definition** — it supplants live workspace state with the recorded item set — so it demands the full acknowledgement trio:

```
sigantry deploy run --rollback --to-release <release-A-id> --rollback-force \
  --rollback-runbook-id DRILL-001 \
  --workspace-id "$TUT06_WSID" --source . --environment DEV \
  --params parameters.yml --audit-dir "$AUDIT_DIR"
# expect: re-publish of A's item set, then a third DeployRecord
```

Without `--rollback-force` the command refuses — by design. The `--rollback-runbook-id` threads your incident reference into the audit record, so six months later the ledger still explains *why* production state jumped backwards.

Step 6 — Verify the ledger tells the whole story

```
sigantry release list --audit-dir "$AUDIT_DIR"
# expect: three records -- A, B, and the rollback record pointing back at A
sigantry release show <rollback-release-id> --audit-dir "$AUDIT_DIR"
# expect: provider/rollback metadata + to-release linkage + audit_hash
```

Open the workspace notebook in the portal: the Step 3 edit is gone — content matches Release A.

What to take into production

- **Time yourself.** The promise to your stakeholders is rollback-in-minutes; the drill gives you your real number.
- The drill used `--audit-dir`; production rollbacks use the default ledger so the record sits beside the original deploys.
- Rollback restores the **recorded item set's content**. Items created after release A are not deleted by rollback — combine with orphan cleanup deliberately, not reflexively.
- Known cosmetic quirk: the rollback record's two timestamps may differ by microseconds (evaluated twice); harmless, tracked as accepted debt.

Success checklist

- ☐ three DeployRecords in the drill ledger: A, B, rollback
- ☐ `release diff A B` showed the edited item
- ☐ rollback refused without `--rollback-force`, succeeded with it
- ☐ portal confirms content reverted to A
- ☐ you know your minutes-to-rollback number

Next: [Tutorial 08 — Scheduled drift alerts](#).

Tutorial 08 — Scheduled Drift Alerts to Teams

Goal: the drift check from Tutorial 03 running unattended on a schedule, posting a summary card to a Teams channel whenever a governed workspace drifts — and staying silent when everything matches.

Time: ~30 minutes (most of it webhook + secret setup). **Builds on:** [Tutorial 03](#) (a manifest with a clean drift baseline, committed to the repo your CI runs from).

The moving parts

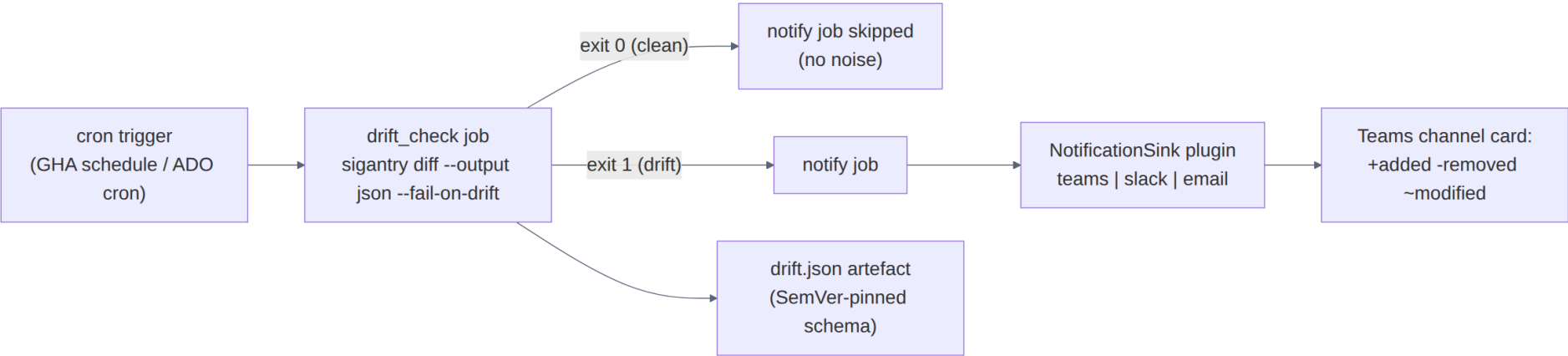


Figure 9: Scheduled Drift Alerts to Teams

The workflow pair ships in the repo: `.github/workflows/drift-check.yml` (GitHub Actions) and `templates/schedules/drift-check.yml` (the Azure DevOps half — same behaviour, kept in lockstep by the dual-CI parity gate).

Step 0 — Pick the right workspace (read this twice)

Wire scheduled drift **only against a workspace your manifest fully governs** (e.g. the one from Tutorial 06, or a team workspace where everything is declared). On a shared workspace, every other team's item counts as `+ added` and the alert is permanently red — an alarm that is always ringing is worse than no alarm. If you must watch a partially-governed workspace, consume `drift.json` and alert on `removed / modified` only.

Step 1 — Create the Teams incoming webhook

In Teams: channel -> Manage channel -> Connectors (or Workflows app) -> Incoming Webhook -> name it `sigantry-drift` -> copy the URL. Treat that URL as a secret — anyone holding it can post to your channel.

Step 2 — Store the webhook as a CI secret

GitHub:

```
gh secret set SIGANTRY_TEAMS_WEBHOOK --repo <owner>/<repo> --body "<webhook-url>"
```

Azure DevOps: add `SIGANTRY_TEAMS_WEBHOOK` to the variable group the pipeline uses, marked secret.

Step 3 — Smoke-test the sink locally before involving CI

Cheaper to debug a webhook from your terminal than from a cron log. Use the exact module the workflow's notify job invokes, fed by a real drift JSON:

```
# 1. Produce a drift JSON from your governed manifest (read-only):
sigantry diff --manifest sync.yml --workspace-id "$WSID" \
  --output json --no-hint > /tmp/drift.json

# 2. Post it through the Teams sink:
export SIGANTRY_NOTIFICATION_SINK=teams
export SIGANTRY_TEAMS_WEBHOOK="<webhook-url>"
python -m sigantry_core.sync._notify_main \
  --drift-json /tmp/drift.json --workspace-id "$WSID"
# expect: a line like
#   sigantry_notify_main: posted drift summary for workspace=<id> drift=... (+a -r ~m =u)
# and a card in the Teams channel within seconds
```

Two contract details worth knowing: the sink posts an Adaptive Card by default (set `SIGANTRY_TEAMS_WEBHOOK_FORMAT=messagecard` for legacy Connector webhooks), and the sinks deliberately swallow transport errors at WARN — a broken webhook still exits 0. The CI alarm is owned by `sigantry diff --fail-on-drift`, not by notification delivery. So: no card but exit 0 means check the URL and look for the WARN log line; also confirm outbound HTTPS to `*.webhook.office.com` / `*.logic.azure.com` is open.

Step 4 — Trigger the workflow once by hand

Before trusting the schedule, run the workflow manually with your real inputs:

```
gh workflow run drift-check.yml --repo <owner>/<repo> \
  -f workspaceId=<governed-workspace-guid> \
  -f manifestPath=<path/to/sync.yml-in-repo> \
  -f environment=prod
gh run watch
# expect: drift_check job green; notify job SKIPPED (you are clean from Tutorial 03);
#         drift.json attached as a run artefact
```

Step 5 — Prove the alert path end-to-end

Inject drift exactly as in Tutorial 03 (rename one governed item in the portal), re-run the workflow, and confirm:

- `drift_check` exits 1 (run shows red — correct, that is the alarm),
- `notify` runs and the Teams card arrives with the +/-/~ summary,
- `drift.json` in the artefacts names the drifted item.

Then reconcile (portal undo or `sync apply`) and re-run once more to watch it go green and silent again.

Step 6 — Let the schedule take over

The workflow's `schedule:` block defaults to a daily cron. Adjust frequency to taste (hourly for hot workspaces, daily for stable ones) and merge. From now on the only time you hear about this workspace is when reality stops matching the contract.

Operational notes

- **Auth in CI:** the workflow authenticates via the repo's federated credential / service connection (no secrets in YAML — see the workflow file's env block). The service principal needs Viewer+ on the workspace.
- **Slack or email instead of Teams:** set `SIGANTRY_NOTIFICATION_SINK=slack` (with `SIGANTRY_SLACK_WEBHOOK`) or `email` (with `SIGANTRY_SMTP_*`) — same plugin seam, same card content.

- The drift JSON schema is SemVer-pinned (`docs/reference/drift-schema.json`) — safe to build downstream automation on.

Success checklist

- ☐ local sink smoke test posted a card
- ☐ manual run: clean workspace -> notify skipped, drift.json artefact present
- ☐ injected drift -> red run + Teams card naming the item
- ☐ reconciled -> green run, silence
- ☐ schedule merged

Next: [Tutorial 09 — PR review bot.](#)

Tutorial 09 — PR Review Bot for Semantic-Model and Lakehouse Diffs

Goal: schema changes made visible in code review. You will run the bot locally against two checkouts and see the Markdown diff it produces, then wire it into CI so every pull request that touches a `.tmdl`, `.platform`, or Lakehouse metadata file gets an automatic review comment.

Time: ~25 minutes. **Builds on:** [Tutorial 01](#).

The problem this solves

Raw git diffs of TMDL files are readable but unreviewed in practice — a renamed measure or a dropped Lakehouse column hides in line noise and gets caught by downstream breakage instead of by a reviewer. The bot parses both sides and posts a *semantic* summary: measures/columns/tables added, removed, changed.

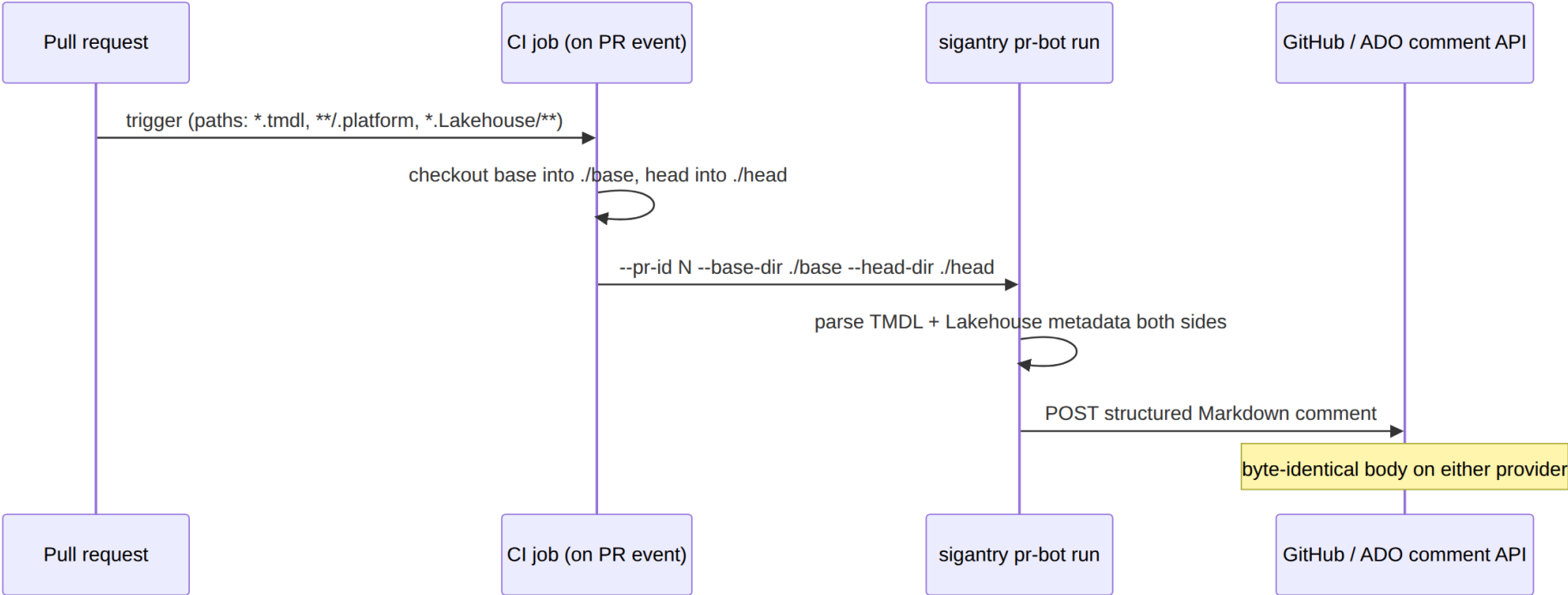


Figure 10: PR Review Bot for Semantic-Model and Lakehouse Diffs

Step 1 — Diff locally with a synthetic change (zero credentials)

The diff engine is importable on its own — simulate a PR in two minutes. **Indentation matters: TMDL uses spaces, not tabs.** A tab-indented file parses as one opaque block and diffs as "no changes" — the most common first-run mistake.

```
mkdir -p ~/sigantry-tut09/{base,head}/model.SemanticModel/definition && cd ~/sigantry-tut09
cat > base/model.SemanticModel/definition/orders.tmdl <<'EOF'
table orders
  column order_id
    dataType: int64
  column amount
    dataType: decimal
  measure 'Total Amount' = SUM(orders[amount])
EOF
cp base/model.SemanticModel/definition/orders.tmdl head/model.SemanticModel/definition/orders.tmdl
cat >> head/model.SemanticModel/definition/orders.tmdl <<'EOF'
  measure 'Order Count' = COUNTROWS(orders)
EOF

python3 - <<'PY'
from pathlib import Path
from sigantry_core.pr_bot.tmdl_diff import diff_tmdl
print(diff_tmdl(Path("base"), Path("head")))
PY
# expect: ... measures_added=[MeasureChange(table='orders', name="'Order Count')] ...
```

This local loop is also how you sanity-check the bot against your real model repos before turning it on: point it at any two checkouts.

A note on `--dry-run`: the CLI's dry-run skips the comment POST but still **fetches PR metadata from the provider API** — so it needs a real PR number, a valid token, and (on GitHub) `GITHUB_REPOSITORY=owner/repo` in the environment. Use the Python loop above for credential-free experimentation; use `--dry-run` to preview the exact Markdown against a real PR.

Step 2 — Understand what it watches

Three surfaces, diffed semantically:

Surface	Files	What gets reported
Semantic models	<code>*.tmdl</code>	tables / columns / measures added, removed, changed
Item identity	<code>**/.platform</code>	item renames, type changes
Lakehouse	<code>*.Lakehouse/**</code> metadata files	table/schema-level changes

Everything else in the PR is ignored — the bot complements, not replaces, human review.

Step 3 — Post for real once

Pick (or open) a scratch PR in a repo you own, then run with a token:

```
export GITHUB_TOKEN=<pat-with-repo-scope>      # or rely on the CI-injected token later
export GITHUB_REPOSITORY=<owner>/<repo>        # the bot reads the repo coords from env
sigantry pr-bot run --provider github --pr-id <number> \
  --base-dir <path-to-base-checkout> --head-dir <path-to-head-checkout>
# expect: exit 0; the PR now carries a structured "Sigantry PR review" comment
# (add --dry-run first to print the exact Markdown without posting)
```

On Azure DevOps the same command takes `--provider ado` with the ADO env vars the pipeline provides; the posted body is byte-identical across providers by contract.

Step 4 — Wire it into CI

The trigger workflow ships in the repo — `.github/workflows/sigantry-pr-bot.yml` (GHA) and `templates/pr-review/sigantry-pr-bot.yml` (ADO). The essential shape, if you are adapting it to a consumer repo:

```
on:
  pull_request:
    paths: ["**/*.tmdl", "**/*.platform", "**/*.Lakehouse/**"]
jobs:
  pr-bot:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
        with: { ref: "${{ github.event.pull_request.base.sha }}", path: base }
      - uses: actions/checkout@v6
        with: { ref: "${{ github.event.pull_request.head.sha }}", path: head }
      - run: pip install sigantry-core # or install from your checkout
      - run: |
          sigantry pr-bot run --provider github \
            --pr-id "${{ github.event.pull_request.number }}" \
            --base-dir ./base --head-dir ./head
    env:
      GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
```

The path filter means the bot only spends CI minutes on PRs that touch model or Lakehouse surfaces.

Step 5 — Prove the no-change behaviour

Open a PR that touches only a README. The bot either does not trigger (path filter) or, if invoked anyway, posts a "no model/Lakehouse changes" comment and exits 0 — reviewers learn to trust its silence as a signal too.

Troubleshooting

Symptom	Cause	Fix
diff reports no changes on a file you edited	tab-indented TMDL	re-indent with spaces (TMDL spec)
<code>GitHub provider requires --token</code> or <code>GITHUB_TOKEN</code>	no token, even on dry-run	export <code>GITHUB_TOKEN</code> (dry-run still reads PR metadata)
<code>GITHUB_REPOSITORY</code> env var must be <code>'owner/repo'</code>	running outside GHA	export it manually; CI sets it automatically
<code>HTTP 401</code> on dry-run	dummy/expired token	dry-run needs a REAL token; use the Python loop for credential-free testing

Exit codes (for CI wiring)

Code	Meaning
0	comment posted (or dry-run printed, or clean no-change comment)
1	validation / auth misconfiguration (missing token, bad flags)
2	REST or auth runtime error
3	unexpected exception

Success checklist

- [] dry-run printed a semantic diff naming the added measure
- [] a real PR carries the bot's structured comment
- [] CI posts automatically on a `.tmdl`-touching PR
- [] a docs-only PR produces no schema-diff noise

Next: [Tutorial 10 — Governance sweeps](#).

Tutorial 10 — Governance Sweeps: RBAC, Labels, Tenant Settings

Goal: three pieces of compliance evidence produced on demand: a workspace RBAC export an auditor can open in Excel, a sensitivity label applied across every item in a workspace, and a hash-stamped tenant-settings baseline you can diff month on month.

Time: ~20 minutes. **Builds on:** [Tutorial 01](#). Tenant-settings export additionally requires a **Fabric administrator** identity; label-sync requires the label GUID from your Purview / compliance portal.

Why these three together

They answer the three questions every audit asks: *who can touch what* (RBAC), *is data classified* (labels), and *what guardrails were configured* (tenant settings) — each with evidence you can hand over rather than screenshots.

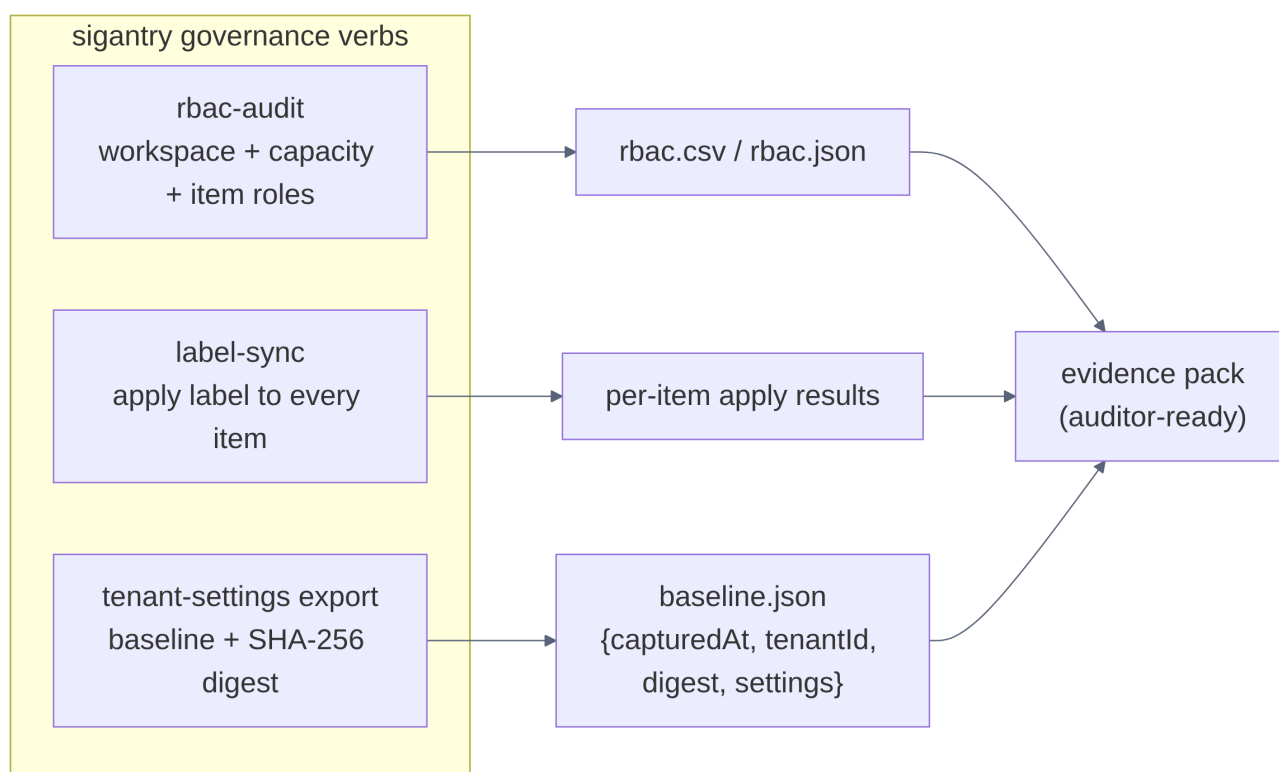


Figure 11: Governance Sweeps: RBAC, Labels, Tenant Settings

Part A — RBAC audit

By default the audit is **tenant-wide**: it sweeps every workspace your identity can see (plus capacity and item layers). Two flags make it actionable day-to-day (added in 3.2.0 — if `sigantry rbac-audit --help` does not show them, upgrade):

- `--workspace-id <guid>` / `-w` (repeatable) scopes the sweep to named workspaces; the capacity layer then narrows to the capacities those workspaces actually run on. A typo'd id fails loudly with `not found (404)` rather than producing an empty-but-green report.
- `--out-dir <dir>` files the rows as a UTC-dated `rbac-audit-<YYYYMMDDTHHMMSSZ>.csv` (or `.json` with `--output json`) and prints a one-line summary instead of dumping rows to the console. `--out <file>` writes an exact path instead; the two are mutually exclusive.

```
# The filed, scoped form -- the shape you act on later:
sigantry rbac-audit -w "$WSID" --out-dir ./access-reviews
# expect a single summary line (live-verified 2026-06-11 against COE_F_SBDEVOPS_POC,
# 18 rows):
#   wrote access-reviews/rbac-audit-20260611T181719Z.csv (rows=18, scope=1 workspace(s))

# The classic tenant-wide console stream still works:
sigantry rbac-audit --output csv > /tmp/rbac.csv      # may take minutes on a big tenant
head -3 /tmp/rbac.csv
# expect:
#   layer,resource_id,resource_name,principal_id,principal_type,principal_name,role,access_status
#   workspace,<guid>,<workspace name>,<guid>,Group,<group name>,Viewer,ok
#   workspace,<guid>,<workspace name>,<guid>,User,<person>,Viewer,via-group:<group name>
```

Note the `via-group:` rows — the audit expands group memberships, so you see the *people* who hold access through a group, not just the group. That is usually the column auditors actually ask about.

Find the admins in a (scoped or tenant-wide) export:

```
grep ",$WSID," /tmp/rbac.csv | awk -F, '$7=="Admin"'
```

(Names containing commas are CSV-quoted, which defeats naive `-F,` splitting — for anything beyond eyeballing, parse the `--output json` form instead.)

`--output json` gives the same rows structured. Practical uses:

- diff this month's export against last month's to spot privilege creep — the dated `--out-dir` files are designed for exactly this comparison;
- review every `Admin` row and confirm each is justified;
- attach the CSV to the access-review ticket — done.

Notes on partial access: capacities your identity cannot read appear as rows with `access_status="forbidden-admin-only"`, and item-level scope appears as `not-accessible-via-rest` placeholders where the REST surface does not expose assignments — the sweep records what it could not see instead of failing.

Part B — Sensitivity label sweep

You need a label GUID from your compliance portal (Purview -> Information protection -> Labels -> copy the label id).

This writes to every item in the workspace — run it first against the Tutorial 06 scratch workspace, not a shared one:

```
sigantry label-sync --workspace-id "$TUT06_WSID" --label-id "<label-guid>" --output json
# expect: per-item results -- each item id with applied/skipped/failed status
```

What to check in the output: `failed` entries usually mean the item type does not support labels (some types cannot carry them) or the principal lacks label-apply permission. Open one item in the portal and confirm the label badge shows.

Service-principal nuance: label APIs behave differently for SPNs vs users; the verb auto-detects from your credential, with `--sp` / `--user` overrides when the detection guesses wrong.

Part C — Tenant-settings baseline

Requires Fabric admin — without it the call fails fast with `AuthError: HTTP 403` (that is your identity lacking the admin API, not a toolkit problem). The export sorts all settings, computes a SHA-256 digest over the canonical JSON, and stamps capture time + tenant:

```
sigantry tenant-settings export --output /tmp/tenant-baseline-$(date -u +%Y%m).json
python3 - <<'PY'
import glob, json
path = sorted(glob.glob("/tmp/tenant-baseline-*.json"))[-1]
b = json.load(open(path))
print("capturedAt:", b["capturedAt"], "| settings:", len(b["settings"]), "| digest:", b["digest"][:16])
PY
# expect: capture timestamp, settings count, digest prefix
```

The monthly ritual that makes this valuable:

```
# next month, export again, then:
python3 - <<'PY'
import glob, json
a, b = [json.load(open(p)) for p in sorted(glob.glob("/tmp/tenant-baseline-*.json"))[-2:]]
print("digests equal:", a["digest"] == b["digest"])
if a["digest"] != b["digest"]:
    na = {s["settingName"]: s for s in a["settings"]}
    for s in b["settings"]:
        if s != na.get(s["settingName"]):
            print("changed:", s["settingName"])
PY
```

Equal digests = provable "nothing changed". Unequal = a named list of exactly which guardrails moved, with two timestamped files as evidence. Commit the baselines to a governance repo and the history writes itself.

Success checklist

- [] `/tmp/rbac.csv` opens with principal/role rows for your workspace
- [] scratch workspace items show the applied label in the portal
- [] baseline JSON has `capturedAt`, `tenantId`, `digest`, `settings[]`
- [] you can articulate the monthly digest-compare ritual

You have now exercised every capability tier. From here: runbooks for terse operational references, CAPABILITIES.md for the full verb inventory, and the CI/CD template pair to put the whole loop — deploy, test, approve, promote, drift-watch — on rails.

Tutorial 11 — Environments and Libraries

Goal: understand exactly how a Fabric **Environment** consumes a custom Python wheel, push one wheel to an Environment with a single command, and know precisely when the notebooks attached to it start importing the new version. This is the third plane Sigantry governs — alongside item *topology* (`sync apply`) and item *content* (`deploy run`), this is the **runtime library** plane.

Time: ~20 minutes. **Builds on:** [Tutorial 01](#) — install + authenticated session. **Uses:** `sigantry env sync`.

The `env sync` flow below mutates a live Environment. The command outputs in this tutorial were dry-run-verified locally on 2026-06-14; the live publish/rebuild behaviour is Fabric's own Environment mechanism, described honestly rather than re-timed here.

The mental model

A **wheel does not flow from a feed straight into a workspace**. Fabric Environments take *uploaded* custom libraries; their "public libraries" source is public PyPI/conda, not an authenticated private ADO feed. So the path always has an upload hop:



Figure 12: Environments and Libraries

Three facts that decide everything downstream:

1. **The Environment is the unit of sharing.** Every notebook attached to a given published Environment imports the same library set. Republish the Environment once and *all* of its notebooks benefit — you never sync notebook-by-notebook.
2. **Publish is a rebuild, not a copy.** Uploading a wheel stages it; the `staging/publish` step rebuilds the Spark image and takes minutes. `env sync` **blocks until that finishes** (it polls `publishDetails.state` to `Success`), so a green `env sync` means the library is genuinely importable — not merely uploaded.
3. **Running sessions do not hot-swap.** A notebook with a live Spark session keeps the old library; it picks up the new one on its **next** session start.

Step 1 — Get a wheel to deploy

Any built wheel works. Build one from a sibling project, or pull a released version from the feed:

```
# from a local project build:
ls dist/*.whl
# expect: e.g. data_platform-1.6.0-py3-none-any.whl

# or pull a pinned release from the feed (CI agents do this):
pip download --no-deps --dest dist "data-platform==1.6.0"
```

Step 2 — Sync the wheel into the Environment

`env sync` is the single-target primitive: one workspace, one Environment, one wheel. You need the **workspace GUID** and the **Environment GUID** (the Environment must already exist — `env sync` updates libraries, it does not create Environments):

```
sigantry env sync \
  --workspace-id <workspace-guid> \
  --environment-id <environment-guid> \
  --wheel dist/data_platform-1.6.0-py3-none-any.whl
# expect (after the publish blocks to completion):
# {
#   "wheelName": "data_platform-1.6.0-py3-none-any.whl",
#   "stagingUploadStatus": "...",
#   "publishLroStatus": "Success",
#   "installedLibraryName": "data_platform-1.6.0-py3-none-any.whl"
# }
```

`publishLroStatus: "Success"` is the load-bearing line: the rebuild finished and the wheel is in the Environment's *published* library set. If the build fails the command raises (non-zero) instead of falsely reporting success.

Step 3 — Confirm a notebook actually imports it

Attach a notebook to that Environment (Fabric portal → notebook → Environment selector), **start a fresh session**, and run:

```
import data_platform
print(data_platform.__version__) # expect: 1.6.0
```

If you had a session open *before* Step 2, restart it — that is the session-boundary rule from the mental model, and it is the single most common "but I deployed it!" confusion.

What to take into production

- **One Environment, many notebooks.** Standardise: point a whole workspace's notebooks at one Environment (e.g. `data-platform-env`). Then a library update is *one* publish, and every notebook benefits on next session — no per-notebook work.
- **Budget for the rebuild.** Each publish costs minutes. Don't publish per-notebook or per-PR; publish per *release*.
- **You cannot point the Environment at the private feed.** The download → upload → publish hop is mandatory (as of 2026-06); that is *why* `env sync` exists. If Microsoft adds private-feed support to Environments later, revisit this.
- **Pre-create the Environment.** `env sync` will not create one; bootstrap it once (portal or `deploy run` of an `*.Environment` item) before the first sync.

Success checklist

- [] you can state where a wheel physically lives before and after `env sync`
- [] `env sync` returned `publishLroStatus: "Success"`
- [] a freshly-started notebook session imported the new version
- [] you can explain why a *running* session didn't see the change

Next: [Tutorial 12 — Config-driven auto-update](#) — turn this one-Environment command into a fleet you keep current from a single file.

Tutorial 12 — Config-Driven Auto-Update

Goal: keep *many* Fabric Environments current from a single `environments.yml`, so that one successful release fans the new wheels out to every Environment that should have them — with the risk controls (gating, pinning, idempotency, fail-isolation) expressed as **data**, not as copy-pasted pipeline shell. Then wire it to fire automatically on release, *safely*.

Time: ~30 minutes. **Builds on:** [Tutorial 11](#) — the single-Environment `env sync` and the consumption model. **Uses:** `sigantry env sync-all`.

Every `env sync-all` invocation in this tutorial (including the JSON output blocks) was executed locally on 2026-06-14 in `--dry-run` mode, which performs no tenant calls. The live publish behaviour is the same `env sync` mechanism from Tutorial 11.

Why a manifest, not a loop

The naive answer to "auto-update every workspace" is a bash loop over `env sync`. That works until the day it matters: it has no idea which Environments are *production*, it re-publishes things that are already current (minutes wasted each), and one failure aborts the rest. `env sync-all` moves those decisions into a versioned file and a tested orchestrator.

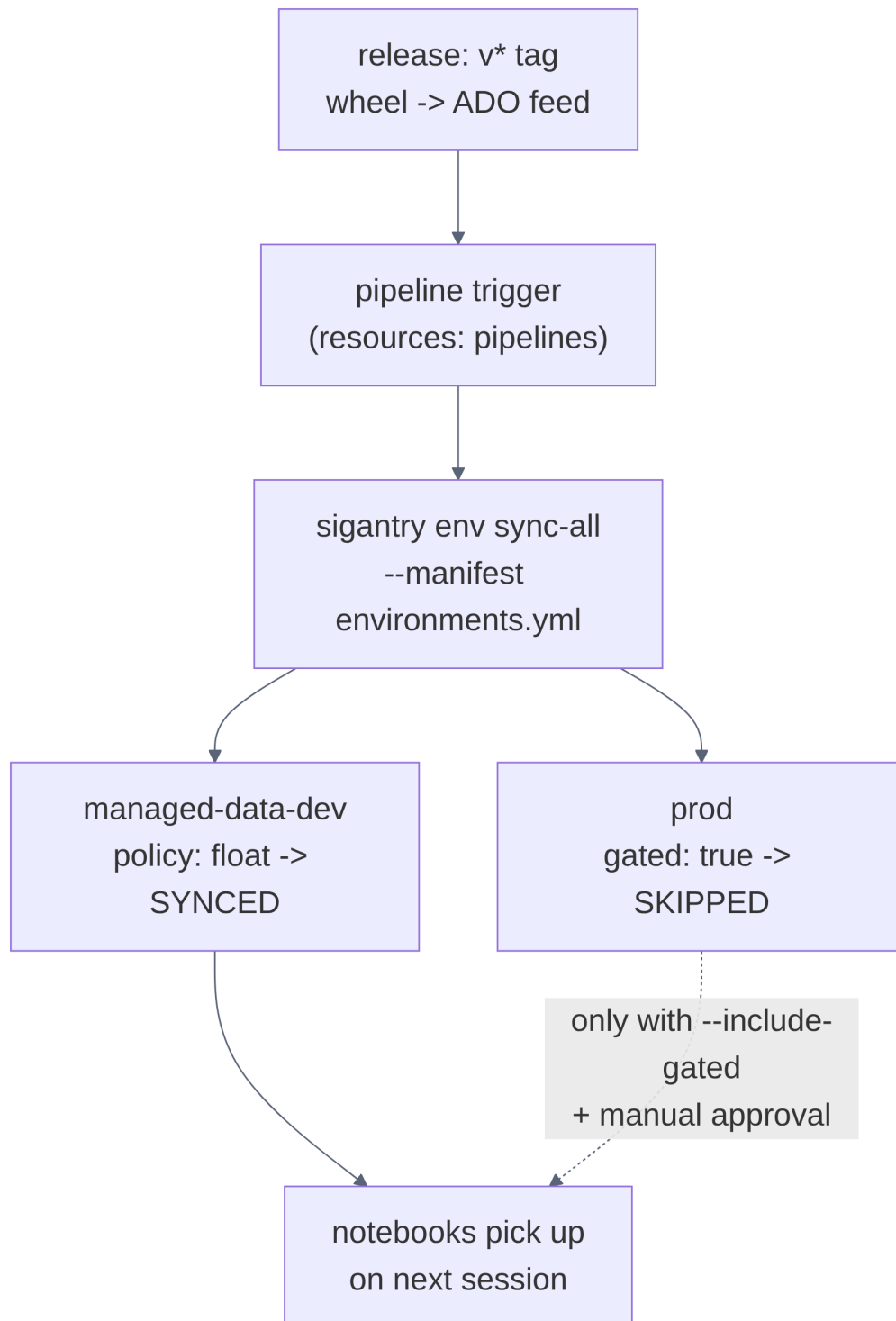


Figure 13: Config-Driven Auto-Update

Step 1 — Write `environments.yml`

```

schema_version: "1.0"
targets:
  - name: managed-data-dev
    workspace_id: <dev-workspace-guid>
    environment_id: <dev-environment-guid>
    wheels: ["dist/*.whl"]      # float: take whatever the release built
    policy: float

  - name: prod
    workspace_id: <prod-workspace-guid>
    environment_id: <prod-environment-guid>
    wheels:                    # pin: exact files, no globs allowed
      - "dist/data_platform-1.6.0-py3-none-any.whl"
      - "dist/data_quality-2.2.0-py3-none-any.whl"
    policy: pin
    gated: true                # never touched unless --include-gated

```

Four risk controls, all as data:

Field	What it buys you
<code>policy: float</code>	DEV tracks the latest build (globs allowed).
<code>policy: pin</code>	PROD must name exact wheels — globs are rejected at load , so a prod target can never silently ship "latest".
<code>gated: true</code>	The target is skipped unless the operator passes <code>--include-gated</code> . An automatic on-release run keeps DEV current and leaves PROD untouched.
(orchestrator)	A wheel already published is skipped (no needless rebuild); one target failing does not abort the rest.

Step 2 — Dry-run first, always

`--dry-run` validates the manifest, resolves every wheel pattern, and prints the plan **without a single tenant call**:

```

sigantry env sync-all --manifest environments.yml --dry-run

```

```
{
  "ok": true,
  "summary": {
    "targets": 2,
    "wheelsSynced": 0,
    "wheelsSkipped": 0,
    "wheelsFailed": 0,
    "targetsGatedSkipped": 1
  },
  "targets": [
    {
      "name": "managed-data-dev",
      "action": "processed",
      "wheels": [
        { "wheel": "data_platform-1.6.0-py3-none-any.whl", "action": "dry-run" },
        { "wheel": "data_quality-2.2.0-py3-none-any.whl", "action": "dry-run" }
      ]
    },
    { "name": "prod", "action": "gated-skipped", "wheels": [] }
  ]
}
```

Read the plan before you trust the run: DEV would sync both wheels; PROD is `gated-skipped`. If a wheel path is wrong you get a `failed` target here, offline, instead of half-way through a live deploy.

Step 3 — Run it for the non-gated targets

```
sigantry env sync-all --manifest environments.yml
# expect: same JSON, but DEV wheels now "synced" with installedLibraryName set;
#         PROD still "gated-skipped". Exit 0 because nothing failed.
```

Run it a **second** time immediately:

```
sigantry env sync-all --manifest environments.yml
# expect: DEV wheels now "skipped-unchanged" — already published, no rebuild.
```

That idempotency is what makes an on-release trigger cheap: re-running costs nothing when nothing changed.

Step 4 — Production is a deliberate, separate act

PROD only moves when a human opts in:

```
sigantry env sync-all --manifest environments.yml --include-gated
# acts on gated targets too — run this behind a manual approval, never on auto-trigger.
```

Because `prod` is `policy: pin`, this deploys *exactly* the named versions. If `dist/data-platform-1.6.0-...whl` isn't present, that target fails loudly (and in isolation) rather than shipping something else.

Step 5 — Wire the automatic trigger (safely)

Make the deploy pipeline fire when the publish pipeline succeeds — but keep the auto path pointed at non-gated targets only. In Azure DevOps:

```
# azure-pipelines-fabric-deploy.yml (consumer pipeline, e.g. in your data-platform repo)
resources:
  pipelines:
    - pipeline: libPublish
      source: lib-publish-ci           # the pipeline that publishes wheels on v*
      trigger:
        tags: [ 'v*' ]              # only real releases, not every build

# ... in the deploy job, after pip-downloading the wheels into dist/:
steps:
  - script: |
      sigantry env sync-all --manifest environments.yml
      # NOTE: no --include-gated. PROD targets stay gated; the auto path only
      # advances DEV/SANDBOX. PROD runs as a separate, manually-approved stage.
      displayName: 'Fan wheels out to non-gated Environments'
```

The result is the goal from the architecture question: **every successful release auto-advances the Environments that opted in (DEV/managed-data), and their notebooks pick the change up on next session** — while production stays a deliberate act.

What to take into production

- **Never auto-push latest to PROD.** Keep prod targets `gated: true` + `policy: pin`, and run `--include-gated` only behind an approval gate. The manifest makes this the default, not a discipline you have to remember.
- **Mind the fan-out cost.** Each non-skipped target publishes serially (minutes each). Keep the auto-trigger target list small; let idempotency absorb the rest.
- **The summary is machine-readable.** `wheelsSynced` / `wheelsFailed` / `targetsGatedSkipped` are stable keys — gate the pipeline on them or post them to Teams (see [Tutorial 08](#) for the notification pattern).
- **--fail-fast is for interactive debugging.** In automation, prefer the default (isolate + continue) so one broken Environment doesn't strand the others; the non-zero exit still flags the run.

Success checklist

- [] `--dry-run` showed your DEV target processed and PROD `gated-skipped`
- [] a real run synced DEV; an immediate re-run reported `skipped-unchanged`

- [] you confirmed PROD only moves with `--include-gated`
- [] your auto-trigger omits `--include-gated` (PROD stays manual)
- [] you can point to where pin-vs-float and gating live (the manifest, not the pipeline)

Next: revisit [Tutorial 07 — Rollback](#) — libraries and item content roll back by different mechanisms; know both.